

```
"""
```

```
BAI2 file generator - Mittera cash management feed.
```

```
Pulls account summary + transaction detail directly from Couchbase via N1QL
(no intermediate files), builds a BAI2 file per the locked spec below, and
writes it to disk. Intended to be triggered by Stonebranch as a plain
Python script (no pandas dependency - all plain dicts/lists).
```

```
RECORD SPEC (as agreed):
```

- 01 - sender=routing number, receiver=hardcoded, date/time=runtime,
file id=1, PRL/block size blank, version=2
- 02 - receiver=same as 01 receiver, originator=same as 01 sender,
group status=1, as-of-date=from query, as-of-time=same as 01 time,
currency=USD, as-of-date modifier=2
- 03 - account + currency from query, summary codes 010/015/100/400 only,
each as "type,amount,,,"
- 16 - type code, amount, funds type (hardcoded "Z" for this partner),
bank ref, customer ref - NO text (moved entirely to 88)
- 88 - tran_desc1-10 joined by comma, wrapped at physical length 94
(content chunk = 91 chars; see wrap_record)
- 49/98/99 - control total = sum of all 03 summary amounts + sum of all
16 amounts; record counts = inclusive physical line counts per
scope. Validated against a real Volante sample file.

```
STILL PLACEHOLDER / NEEDS YOUR INPUT:
```

- Couchbase connection (see get_couchbase_cluster) - replace with your
standard connection snippet.
- BAI2_CONFIG sender_id / receiver_id - your real routing number and
the hardcoded receiver ID.
- The two N1QL query strings - update to match your corrected field
names/query if anything changes from what's implemented here.

```
"""
```

```
import sys
```

```
from datetime import datetime
```

```
# -----
# CONFIG - replace placeholders with real values
# -----
```

```
BAI2_CONFIG = {
```

```
    "sender_id": "REPLACE_WITH_ROUTING_NUMBER",    # bank routing number
```

```
    "receiver_id": "REPLACE_WITH_RECEIVER_ID",    # hardcoded receiver ID
```

```

        "file_id_number": "1",
        "group_status": "1",
        "currency_code": "USD",
        "as_of_date_modifier": "2",
        "version_number": "2",
        "physical_record_length": 94, # wrap width for 88 continuation lines
    }

ACCOUNT_ROUTING_NUMBER = "xxxxxxx|07109999" # PLACEHOLDER - Mittera's account|ro
POSTING_DATE = None # PLACEHOLDER - set to 'YYYY-MM-DD' or leave None to use tod

# -----
# COUCHBASE CONNECTION - PLACEHOLDER, replace with your standard snippet
# -----

def get_couchbase_cluster():
    """
    PLACEHOLDER. Replace with your standard Couchbase connection snippet
    (Key Vault-sourced connection string / credentials, per your existing
    pipeline convention). Must return an object with a .query(nlql_string)
    method compatible with the couchbase Python SDK.
    """
    raise NotImplementedError("Wire up your standard Couchbase connection here")

def run_nlql(cluster, query_string):
    """Runs an N1QL query and returns a list of result rows (as dicts)."""
    result = cluster.query(query_string)
    return [row for row in result]

# -----
# N1QL QUERIES - update if your corrected field names differ
# -----

def build_summary_query(account_routing_number, posting_date):
    return f"""
    select
    d.acct_nb,
    acct_sts.opng_ldgr_bal.bai_cd as opng_ldgr_bai_cd,
    acct_sts.opng_ldgr_bal.amt as opng_ldgr_bal,
    acct_sts.clsq_ldgr_bal.bai_cd as clsq_ldgr_bai_cd,
    acct_sts.clsq_ldgr_bal.amt as clsq_ldgr_bal,
    acct_sts.ttl_cdt_amt.bai_cd as ttl_cdt_amt_bai_cd,
    acct_sts.ttl_cdt_amt.amt as ttl_cdt_amt,
    acct_sts.ttl_dbt_amt.bai_cd as ttl_dbt_amt_bai_cd,
    """

```

```

    acct_sts.ttl_dbt_amt.amt as ttl_dbt_amt
    from semantic.`dhub_standard_reports`.`daily_statement_detail_priordays` AS d
    where d.acct_rtg_nb = "{account_routing_number}"
    and d.pstng_dt = '{posting_date}'
    """

def build_detail_query(account_routing_number, posting_date):
    return f"""
    select
    d.acct_nb as account,
    c.dtl_bai_cd as bai_code,
    'CR' as tran_type,
    REPLACE(c.tx_amt, '.', '') as amount,
    -- bk_ref / cust_ref / tran_desc1-10: per your existing detail query logic
    d.rtg_nb as routing_number
    from semantic.`dhub_standard_reports`.`daily_statement_detail_priordays` AS d
    where d.acct_rtg_nb = "{account_routing_number}"
    and d.pstng_dt = '{posting_date}'
    """

# -----
# BAI2 BUILDING
# -----

def strip_decimal(amount_str):
    """'186455.00' -> '18645500'. BAI2 amounts are unsigned, no decimal point."""
    return str(amount_str).replace(".", "")

def to_yymmdd(iso_date_str):
    """'2026-09-08' -> '260908'."""
    return datetime.strptime(iso_date_str, "%Y-%m-%d").strftime("%y%m%d")

def wrap_record(record_code, body, prl):
    """
    Wrap a logical record's comma-joined body into physical lines of
    exactly `prl` characters (except possibly the last line), prefixing
    the first line with the real record code and every continuation line
    with "88,". Validated against real Volante output.
    """
    chunk_size = prl - 3  # "XX," or "88," is always 3 chars
    lines = []
    i = 0
    first = True

```

```

while i < len(body) or first:
    chunk = body[i:i + chunk_size]
    prefix = f"{record_code}," if first else "88,"
    lines.append(prefix + chunk)
    i += chunk_size
    first = False
    if i >= len(body):
        break
return lines

def build_01(sender_id, receiver_id, file_date, file_time):
    return (
        f"01,{sender_id},{receiver_id},{file_date},{file_time},"
        f"{BAI2_CONFIG['file_id_number']},,,{BAI2_CONFIG['version_number']}/"
    )

def build_02(receiver_id, sender_id, as_of_date, as_of_time):
    return (
        f"02,{receiver_id},{sender_id},{BAI2_CONFIG['group_status']},"
        f"{as_of_date},{as_of_time},{BAI2_CONFIG['currency_code']},"
        f"{BAI2_CONFIG['as_of_date_modifier']}/"
    )

def build_03_lines(summary_row, prl):
    account = summary_row["acct_nb"]
    currency = BAI2_CONFIG["currency_code"]

    summary_pairs = [
        (summary_row["opng_ldgr_bai_cd"], strip_decimal(summary_row["opng_ldgr_ba
        (summary_row["clsg_ldgr_bai_cd"], strip_decimal(summary_row["clsg_ldgr_ba
        (summary_row["ttl_cdt_amt_bai_cd"], strip_decimal(summary_row["ttl_cdt_am
        (summary_row["ttl_dbt_amt_bai_cd"], strip_decimal(summary_row["ttl_dbt_am
    ]

    parts = [account, currency]
    for type_code, amount in summary_pairs:
        parts.extend([type_code, amount, "", ""])
    body = ",".join(parts) + "/"

    summary_amount_sum = sum(int(amt) for _, amt in summary_pairs)
    return wrap_record("03", body, prl), summary_amount_sum

def build_16_88_lines(txn_row, prl):

```

```

bai_code = txn_row["bai_code"]
amount = txn_row["amount"]
bk_ref = txn_row.get("bk_ref", "")
cust_ref = txn_row.get("cust_ref", "")

# 16 line: no text - ends with cust_ref, or ",/" if cust_ref is blank
line_16 = f"16,{bai_code},{amount},Z,{bk_ref},{cust_ref}/"

desc_fields = [txn_row.get(f"tran_desc{i}", "") for i in range(1, 11)]
text_body = ",".join(desc_fields)
lines_88 = wrap_record("88", text_body, prl) # first "88" prefix intentional

return [line_16] + lines_88, int(amount)

def build_bai2(summary_row, transaction_rows):
    prl = BAI2_CONFIG["physical_record_length"]
    now = datetime.now()
    file_date = now.strftime("%Y%m%d")
    file_time = now.strftime("%H%M")

    sender_id = BAI2_CONFIG["sender_id"]
    receiver_id = BAI2_CONFIG["receiver_id"]
    as_of_date = to_ymdd(summary_row["pstng_dt"]) # e.g. "2026-09-08" -> "2609"

    file_lines = []
    file_lines.append(build_01(sender_id, receiver_id, file_date, file_time))
    file_lines.append(build_02(receiver_id, sender_id, as_of_date, file_time))

    account_lines, summary_amount_sum = build_03_lines(summary_row, prl)
    file_lines.extend(account_lines)

    detail_amount_sum = 0
    for txn_row in transaction_rows:
        txn_lines, amount = build_16_88_lines(txn_row, prl)
        file_lines.extend(txn_lines)
        detail_amount_sum += amount

    account_control_total = summary_amount_sum + detail_amount_sum
    file_lines.append(f"49,{account_control_total},{len(file_lines) + 1}/")

    group_total = account_control_total # single account in this group
    group_record_count = len(file_lines) + 1
    file_lines.append(f"98,{group_total},1,{group_record_count}/")

    file_total = group_total # single group in this file
    file_record_count = len(file_lines) + 1

```

```

        file_lines.append(f"99,{file_total},1,{file_record_count}/")

    return file_lines

# -----
# MAIN
# -----

def main():
    posting_date = POSTING_DATE or datetime.now().strftime("%Y-%m-%d")

    cluster = get_couchbase_cluster()

    summary_rows = run_n1ql(cluster, build_summary_query(ACCOUNT_ROUTING_NUMBER,
    if not summary_rows:
        print("No summary row returned - check account/date parameters.")
        sys.exit(1)
    summary_row = summary_rows[0]

    transaction_rows = run_n1ql(cluster, build_detail_query(ACCOUNT_ROUTING_NUMBE

    lines = build_bai2(summary_row, transaction_rows)

    output_path = f"mittera_{posting_date.replace('-', '')}.bai2"
    with open(output_path, "w", encoding="utf-8") as f:
        f.write("\n".join(lines) + "\n")

    print(f"Wrote {output_path}: {len(lines)} lines")

if __name__ == "__main__":
    main()

```